

README_AI.md

Project Overview

This is a Car Rental Management System built with Angular.

The application follows a hybrid Angular architecture:

- Application starts using `bootstrapApplication()`
- Loads `AppModule`
- Uses standalone components
- Uses lazy loading extensively
- Uses domain-driven module separation

Main business domains:

- Rent
- Finance
- Maintenance

Folder Structure

```
src/app
├─ app.component.ts
├─ app.module.ts
├─ app.routes.ts
├─
├─ core/
├─ shared/
├─ auth/
├─
└─ modules/
    ├─ rent/
    ├─ finance/
    └─ maintenance/
```

Architecture Layers

core

Contains:

- Authentication state
- User session
- Permission constants
- API response contracts
- Core interfaces

Examples:

```
core/  
├─ auth/  
├─ interfaces/  
└─ constants/
```

shared

Reusable across all modules.

Contains:

- Layout
- Header
- Sidebar
- Breadcrumb
- Guards
- BaseService
- Shared UI Components
- Common dialogs
- Toast notifications

Examples:

```
shared/  
├─ services/  
├─ guards/  
├─ component/  
└─ ui/
```

auth

Authentication pages only.

Examples:

```
auth/login
auth/register
auth/401
auth/403
auth/404
```

Authentication pages do NOT use the main application layout.

modules

Business logic is separated by domain.

```
modules/
├─ rent/
├─ finance/
└─ maintenance/
```

Each module contains:

```
components/
services/
models/
common/
utils/
```

Routing Architecture

Root Routing

Defined in:

```
app.routes.ts
```

Flow:

```

/
└─ redirect → auth/login

/auth/*
└─ AuthenticationModule

Authenticated User
└─ Content Layout
    ├── FinanceModule
    ├── RentModule
    └─ MaintenanceModule

```

Important Routing Rule

All feature modules are mounted under:

```
path: ''
```

Example:

```

{
  path: '',
  loadChildren: () => import('./modules/rent/rent.module')
}

```

This means:

```

/customers
/bookings
/vehicles

NOT

/rent/customers
/rent/bookings

```

There is NO module prefix.

Layout Structure

Main layout component:

```
shared/component/layout/content/
```

Contains:

- Header
- Sidebar
- Breadcrumb
- Router Outlet

Example:

```
<div class="page-body">  
  <router-outlet />  
</div>
```

Feature Organization

Every feature follows:

```
feature-name/  
├─ feature-list/  
├─ feature-form/  
└─ feature-details/
```

Example:

```
customers/  
├─ customer-list/  
├─ customer-form/  
└─ customer-details/
```

Component Conventions

List Component

Purpose:

- Table
- Search
- Pagination
- Filters

Example:

```
customer-list.component.ts
```

Form Component

Purpose:

- Create
- Update

Example:

```
customer-form.component.ts
```

Details Component

Purpose:

- Read-only view

Example:

```
customer-details.component.ts
```

Standalone Components

All new components should be:

```
standalone: true
```

Loaded using:

```
loadComponent ()
```

Avoid declaring new pages inside NgModules.

Service Architecture

Always follow:

```
Component
↓
Domain Service
↓
BaseService
↓
HttpClient
↓
Backend API
```

Never call HttpClient directly from components.

BaseService

All API communication must go through:

```
shared/services/base.service.ts
```

Responsibilities:

- GET
- POST
- PUT
- DELETE
- Error handling
- Result<T> parsing

Backend base URL:

```
environment.apiUrl
```

Domain Services

Example:

```
CustomerService
BookingService
```

```
VehicleService
MaintenanceService
```

Rules:

- Use inject(BaseService)
- No direct HttpClient usage
- One service per business domain

Example:

```
@Injectable({
  providedIn: 'root'
})
export class CustomerService {}
```

Models

Location:

```
modules/*/models/
```

Examples:

```
customer.model.ts
booking.model.ts
vehicle.model.ts
```

Contains:

- Interfaces
- DTOs
- Request models
- Response models

Normalizers

Every API response should be normalized.

Example:

```
customer.normalizer.ts
```

Purpose:

Convert backend JSON into frontend-friendly models.

Example:

```
normalizeCustomer()  
normalizeBooking()  
normalizeVehicle()
```

Guards

Main guards:

```
authGuard  
privilegeGuard
```

authGuard

Checks:

- Logged in user
- Valid session

privilegeGuard

Checks:

```
route.data.privileges
```

Against current user permissions.

Privileges

Privileges are defined in:

```
core/constants
```

Example:

```
APP_PRIVILEGES.customer  
APP_PRIVILEGES.booking  
APP_PRIVILEGES.vehicle
```

Routes should declare required privileges.

Example:

```
data: {  
  privileges: [APP_PRIVILEGES.customer]  
}
```

Feature Route Pattern

Standard CRUD routes:

```
/customers  
/customers/create  
/customers/:id/edit  
/customers/:id/details
```

Special actions:

```
/booking/:id/finish
```

Coding Rules

Services

✓ Use BaseService

✗ Do not use HttpClient directly

Components

✓ Standalone

✓ Lazy loaded

✗ Avoid NgModule declarations

Models

✓ Strong typing

✗ Avoid any

Routing

✓ Use loadComponent

✓ Use lazy loading

✗ Do not eagerly load pages

Shared

Put code in shared only if reused by multiple domains.

Examples:

```
DatePicker
Pagination
BaseService
ToastService
ConfirmDialog
```

Maintenance Module Notes

Maintenance module follows the same architecture as:

- Rent
- Finance

Structure:

```
maintenance/
├─ components/
```

```
|— services/  
|— models/  
|— common/  
└— utils/
```

Any maintenance feature should follow existing patterns.

Development Checklist

Before creating a new feature:

1. Create model
 2. Create normalizer
 3. Create service
 4. Create standalone component
 5. Add route
 6. Add privileges
 7. Use BaseService
 8. Follow lazy loading pattern
-

Important Rule For AI Agents

When generating code:

- Follow existing folder structure.
- Do not introduce alternative architectures.
- Prefer consistency over innovation.
- Reuse existing shared components.
- Keep naming conventions identical to existing modules.
- Use standalone Angular components.
- Use BaseService for all API communication.
- Respect privilege-based routing.
- Match existing CRUD route patterns.